Module 11

System Design — Multi-Agent Pipelines

Designing reliable, observable, cost-tracked autonomous systems

ContentAutomation2 — Backend Learning Series
Zero Prerequisites → Production-Ready Code

1. What is System Design?

System design is the process of defining the architecture, components, and data flows of a software system. It answers: how do the pieces fit together? What happens when something fails? How does the system scale? For ContentAutomation2, the key question was: how do you automate a content pipeline that runs daily, involves LLMs (which are slow and expensive), and requires human approval at key steps?

2. Event-Driven Architecture

In an event-driven system, components communicate by publishing and reacting to events, not by directly calling each other. In ContentAutomation2:

- Research agent completes → publishes an event → Scoring agent starts automatically
- Draft approved → event → Publishing agent picks it up
- Post published → event → Analytics agent scheduled for 24h/72h/7d later

This decoupling means each agent can fail, retry, or scale independently. arq + Redis is the event bus here — 'enqueue_job' IS publishing an event.

3. Pipeline Architecture — The 6 Agents

```
DAILY PIPELINE:
  [Cron 6 AM IST]
      → Research Agent
          scrapes 7 news sites
          scores headlines with Claude Haiku
          extracts articles with Playwright
          summarises with Claude Sonnet
          writes to raw_content
      → [auto-chains] → Scoring Agent
          reads unprocessed raw_content
          generates content ideas per platform
          checks for recent coverage overlap (vector search)
          writes pending ideas
      → [Gate 1: Human approves ideas]
      → [Orchestrator trigger] → Creation Agent
          reads approved ideas
          retrieves brand voice examples (vector search)
          retrieves KB context (vector search)
          generates drafts with Claude Sonnet
      → [Gate 2: Human approves drafts]

EVERY 15 MINUTES:
  [Cron] → Publishing Agent
      posts approved, due drafts to LinkedIn/Twitter/WordPress/Email
      schedules analytics jobs at 24h, 72h, 7d

AT 24h, 72h, 7d:
  → Analytics Agent
      fetches metrics from platform APIs
      at 7d: updates style_guide and topic_performance_model
```

4. Human-in-the-Loop Design

Fully automated AI content is risky — especially for regulated financial content. ContentAutomation2 has **two mandatory human approval gates**:

- Gate 1 (Ideas): Human reviews the idea angle, platform, and source. Can edit angle before approving.
- Gate 2 (Drafts): Human reviews full content. Finance-flagged items (company names, investment advice) CANNOT be auto-approved.

This pattern is called Human-in-the-Loop (HITL). It balances automation speed with human oversight for quality and legal compliance.

5. Circuit Breakers — Auto-Pause on Repeated Failures

```
# If a news site fails 5 consecutive times, auto-pause it:
def record_site_failure(supabase, site_id, error_msg, threshold=5):
    # Increment consecutive_failures
    supabase.table("curated_sites").rpc("increment_failures", {"site_id": str(site_id)}).execute(

    # Check if threshold exceeded
    resp = supabase.table("curated_sites").select("consecutive_failures").eq("id", str(site_id)).
    failures = resp.data[0]["consecutive_failures"]

    if failures >= threshold:
        # Soft-pause the site
        supabase.table("curated_sites").update({"active": False}).eq("id", str(site_id)).execute(
        # Alert the team
        send_slack_alert(slack_url, f"Site {site_id} auto-paused after {failures} failures")
```

Note: *ContentAutomation2 tracks consecutive_failures per site. After 5 failures it sets active=false and sends a Slack alert. The human can re-enable it via the admin panel.*

6. Observability — Logging and Run Logs

You can't fix what you can't see. Every agent run writes a structured log to the run_logs table:

- agent_name — which agent ran
- trigger_type — cron, event, manual, or orchestrator
- processed_count, success_count, failure_count — what happened
- duration_seconds — how long it took
- reasoning_trace — why the agent made each decision
- errors — serialised exception details for debugging
- token_cost — input/output tokens and estimated USD per model

7. Cost Control

```
# Every agent accumulates token costs and writes to cost_log table:
# cost_log: agent_name, date, token_count, estimated_cost_usd

# Upsert: if the agent already ran today, add to the existing row
def upsert_cost_log(supabase, agent_name: str, total_usd: float, token_count: int):
```

```
    today = date.today().isoformat()
    supabase.table("cost_log").upsert({
        "agent_name":         agent_name,
        "date":               today,
        "token_count":        token_count,
        "estimated_cost_usd": total_usd,
    }, on_conflict="agent_name,date").execute()

# Alert if a single run costs more than the threshold
if total_usd >= settings.daily_cost_alert_usd:
    send_slack_alert(webhook, f"Cost alert: ${total_usd:.4f}")
```

8. Graceful Degradation

When an external dependency fails, degrade gracefully rather than crashing.

- Embedding API fails → fall back to local fastembed model
- arq Redis unavailable → API returns 503 but does not crash
- Pre-scoring (Claude) fails entirely → proceed with all articles (no score gate)
- Single article fetch fails → log and skip, don't abort the whole site

9. Idempotency — Running the Same Thing Twice

Idempotent operations can be run multiple times with the same result. Critical for job queues where failures may cause retries.

```
# Idempotent insert: use ON CONFLICT DO NOTHING or DO UPDATE
supabase.table("raw_content").upsert({
    "normalized_url": normalize_url(url),
    "title":          title,
    ...
}, on_conflict="normalized_url").execute()
# Second run with same URL → updates the row, doesn't create a duplicate
```

10. Deployment Considerations

- Run FastAPI and arq worker as separate processes (one process each).
- Use a process manager (systemd, Docker, or Railway) to restart crashed processes.
- Redis should be managed (Upstash free tier) — not self-managed for simplicity.
- Store all secrets in environment variables, never in code or git.
- Use Docker to containerise the app for consistent deployment.

11. Practice Exercise — Design Your Own Pipeline

Design a system that:

- Monitors 5 job boards for Python developer job postings every 3 hours
- Scores jobs on relevance (salary, remote, Python focus)
- Sends a daily digest email with the top 10 jobs

Draw the pipeline, identify the agents, define the database tables, choose the trigger mechanism, add a human-in-the-loop step, and plan for failures.